



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

---

# **Software Requirements Specification**

## **Task Manager Application**

Version 1.0 approved

Prepared by Team 4

PES University, Bangalore

PES1UG24CS309 - PES1202400285 - OM SATYAJYOTI PATTANAYAK

PES1UG24CS325 - PES1202400330 - PRACHI GANESH JOSHI

PES1UG24CS344 - PES1202400032 - priyansh soni

PES1UG24CS362 - PES1202403607 - RAJAT KIRAN KOOGGE

02-10-2026



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

## **Table of Contents**

### ***1. Introduction***

- 1.1 Purpose**
- 1.2 Scope**
- 1.3 Definitions, acronyms, and abbreviations**
- 1.4 References**
- 1.5 Overview**

### ***2. Overall description***

- 2.1 Product perspective**
- 2.2 Product functions**
- 2.3 User characteristics**
- 2.4 Constraints**
- 2.5 Assumptions and dependencies**

### ***3. Specific requirements***

- 3.1 External Interface**
- 3.2 Functional Requirements**
- 3.3 Non-Functional Requirements**
- 3.4 Design Constraints**

***Appendixes***

***Index***



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

## Revision History

| Name   | Date       | Reason For Changes                                | Version |
|--------|------------|---------------------------------------------------|---------|
| Team 4 | 02-10-2026 | Initial SRS preparation and requirements baseline | 1.0     |
|        |            |                                                   |         |

## Introduction

### Purpose

*This Software Requirements Specification (SRS) defines the complete software requirements for Version 1.0 of the Task Manager Application. It establishes what the product shall provide to its users, the interfaces through which it shall operate, the functional and non-functional requirements that shall be satisfied, and the constraints under which the system shall be designed and implemented. The SRS covers the complete Task Manager application scope for Team 4 and provides the baseline for architecture, design, implementation, testing, and traceability.*

### Intended Audience

**The intended readers are:**

- Project team members, who use it as the agreed requirements baseline.
- Developers, who use it to implement the frontend, backend, APIs, authentication, task-management services, and data persistence.
- Testers/QA engineers, who derive functional, non-functional, and security test cases from the requirements.
- Project instructors and evaluators, who verify conformance with the Software Engineering mini-project requirements.
- Users/reviewers, who need to understand the capabilities and expected behaviour of the application.

The document is organized from introductory and product-level information, through interfaces and detailed requirements, to system features, non-functional requirements, appendices, and the requirements traceability matrix.

### Product Scope

#### In Scope

- The Task Manager Application is a web-based software product for organizing, assigning, updating, and tracking work items in one centralized system.
- Its scope includes user registration and authentication; creation, viewing, editing, and deletion of tasks; assignment of tasks to registered team members; task status and priority management; search and filtering of authorized tasks; and dashboard-based progress tracking.

#### Objectives and Goals

- The primary objectives are to provide a structured way to manage work, make task ownership and status visible, reduce ambiguity in task tracking, and provide a persistent record of task information.

#### Expected Benefits

- Expected benefits include easier task organization, improved visibility of work progress, controlled access to task information, and a common interface for managing work.

#### Project Alignment

- The product is intended as a software-engineering mini-project and is aligned with the project goal of demonstrating requirements engineering, architecture/design, implementation, and testing of a usable task-management system.

## Out of Scope

- Notifications, advanced collaboration, production cloud administration, and other features not defined in this SRS are outside the approved scope unless added through a later requirements change.

## References

1. Software Engineering Mini-Project Deliverables Part-1, 2026 — course-provided deliverables document; version/date information not specified; source: course project materials.
2. SRS-Document (1).docx — course-provided Software Requirements Specification template; version/date information not specified; source: course project materials.
3. Mini-Project Problem Statement — Section F, Team 4, Task Manager Application; course-provided project material; version/date information not specified.
4. Software Engineering mini-project guidelines/instructions supplied for the project; version/date information not specified.
5. Software Architecture and Design Specification (SAD), Task Manager Application, Team 4, Version 1.0, 02-10-2026.
6. Software Test Plan (STP), Task Manager Application, Team 4, Version 1.0, 02-10-2026.

## Overall Description

### Product Perspective

*The Task Manager Application is a new, self-contained web-based software product developed as a Software Engineering mini-project. It is not specified as a replacement for an existing system or as a component of a larger enterprise system. The product consists of a React.js user interface, a Node.js/Express.js application backend, application APIs, authentication and authorization logic, task-management services, dashboard functionality, and persistent database storage. Users interact with the React interface; the frontend communicates with backend services through REST-style APIs; the backend validates requests, applies business rules and security checks, and reads/writes persistent user and task information. The system therefore has clear boundaries between presentation, application/service logic, and data persistence.*

### Product Functions

The major product functions are:

- Register a new user with a unique email address and password.
- Authenticate registered users and protect task-management functions from unauthenticated access.
- Create tasks containing the information required by the SRS.
- View tasks that the current user is authorized to access.
- Edit and delete authorized tasks.
- Assign tasks to registered team members.
- Set and modify task priority and deadline information.
- Update and track task status.
- Search and filter authorized tasks using task attributes.
- Present task and progress information through a dashboard.

- Validate user input and return appropriate errors for invalid or unauthorized operations.
- Persist successfully stored user and task information for subsequent use.

## User Classes and Characteristics

- **Registered User:** A user who has created an account and can authenticate to the system. The user can perform task operations permitted by the application and can view progress information for authorized tasks.
- **Team Member:** A registered user who may be assigned tasks. A team member can view assigned/authorized tasks and update permitted task information such as status.
- **Task Manager/Task Owner:** An authorized user who creates or manages tasks and may assign them to registered team members. The exact permission model shall follow the authorization rules defined by the implementation and SRS.
- **Project Evaluator/Reviewer:** A non-operational stakeholder who reviews the system against the SRS, architecture, design, and testing deliverables.

The primary operational users are expected to have basic web-application skills. No specialized technical knowledge is required for normal task creation, viewing, editing, assignment, or status tracking.

## Operating Environment

*The application shall operate as a web-based system. On the client side, a modern web browser shall be used to access the React.js frontend. The application backend shall run in a Node.js environment using Express.js for HTTP/API services. Persistent user and task data shall be stored in the database selected for implementation. The client and server shall communicate over HTTP/HTTPS, with HTTPS used for secure deployment. The development environment shall support the required JavaScript/Node.js tooling, package dependencies, Git-based source control, and database services. The application shall not require dedicated client hardware beyond a general-purpose computer capable of running a supported browser.*

## Design and Implementation Constraints

**The following constraints limit implementation choices:**

1. The frontend shall use JavaScript with React.js.
2. The backend shall use Node.js with Express.js.
3. The system shall provide persistent storage for user and task information.
4. Protected operations shall use authentication and authorization.
5. Frontend/backend communication shall use defined application APIs and structured request/response data.
6. User input shall be validated before processing.
7. Passwords and other sensitive authentication information shall not be exposed in normal application responses or logs.
8. The architecture shall remain modular so that major components can be developed and tested independently.
9. Implementation shall follow the approved SRS and SAD rather than introducing unapproved features.
10. The project shall use standard web and software-engineering practices appropriate to the mini-project environment.

## 2.6 Assumptions and Dependencies

### Assumptions

- Users have access to a supported modern web browser and network connectivity to the application.
- Users provide valid information during registration and task operations.
- The development/test environment can provide React.js, Node.js, Express.js, the required package dependencies, and the selected database.

### Dependencies

- The project depends on these software components and libraries being available and compatible during implementation.
- The application also depends on the availability of persistent storage for user and task records and on the team maintaining the agreed API and data contracts.
- If a dependency changes materially, the affected requirements shall be reviewed before implementation or testing continues.

## External Interface Requirements

### User Interfaces

#### The user interface shall provide:

- The user interface shall be a web-based graphical interface with consistent navigation and clearly labelled actions.
- It shall provide interfaces for registration, login, task listing, task creation, task details, task editing, task assignment, status/priority updates, search/filtering, and dashboard/progress viewing.
- Required fields shall be clearly indicated.
- Input validation and meaningful error messages shall be shown near the relevant action.
- The interface shall prevent or clearly communicate invalid operations, unauthorized actions, and failed requests.
- Lists and task details shall present sufficient information to identify a task and understand its current state, priority, deadline, and assignee where applicable.
- The UI shall remain consistent across the major task-management screens.

### Software Interfaces

#### Software interface requirements:

- The React.js frontend shall communicate with the Node.js/Express.js backend through REST-style application APIs.
- API requests and responses shall use JSON or another agreed structured representation.
- The backend shall communicate with the selected database to create, retrieve, update, and delete persistent user and task information.
- Authentication information shall be handled through the backend security mechanism selected during implementation.
- The application shall isolate data-access responsibilities from the presentation layer so that frontend components do not directly manage database operations.

- Detailed endpoint definitions are specified in the SAD document.

## Communications Interfaces

**The communications interface shall satisfy:**

- The system shall use HTTP/HTTPS for client-server communication.
- REST-style API requests shall identify the required resource and operation, and request/response payloads shall use JSON or another agreed structured data format.
- HTTPS shall be used in secure deployment so that credentials and task information are protected while in transit.
- Authentication failures and authorization failures shall be communicated using appropriate HTTP response semantics, without exposing sensitive information.
- Communication shall support normal browser-to-server task-management operations and shall preserve request/response consistency between the React frontend and Node.js/Express backend.

Hardware Interfaces: No dedicated hardware interface is required. The application is intended to run on general-purpose computers used by project users and developers. Standard input devices such as keyboard and mouse/touchpad may be used through the web browser, but no special hardware device, sensor, or controller is required.

## Analysis Models

**The analysis model used in the SRS is:**

- The primary analysis model for the SRS is the UML use case diagram included with this document. It identifies the main actor interaction with the Task Manager Application, including registration, authentication, task creation and management, assignment, status/priority updates, search/filtering, and progress monitoring. The use case model provides a high-level view of system behaviour and establishes a basis for the detailed functional requirements. The detailed component and sequence models are maintained in the Software Architecture and Design Specification.

## System Features

*The functional requirements are organized by major services so that each system feature has a clear description, priority, stimulus/response behaviour, and uniquely identified functional requirements. The feature groups are Authentication and User Management; Task Management; Task Assignment and Tracking; and Search, Filtering, and Dashboard.*

### System Feature 1

#### **Authentication and User Management**

##### **5.1.1 Description and Priority**

*Provides user registration, login, logout/session termination as supported by the implementation, and protection of authenticated functions. Priority: High. This feature establishes the identity and access baseline required before users can perform protected task-management operations.*

##### **5.1.2 Stimulus/Response Sequences**

*Registration stimulus: the user enters the required registration details and submits the form; the system validates the input, checks uniqueness, and creates the account or returns a validation/conflict error. Login stimulus: the user submits credentials; the system validates them and establishes an authenticated session/token or returns an authentication error. Logout/session termination stimulus: the user requests termination of the authenticated session; the system invalidates or ends the applicable client authentication state.*

### **5.1.3 Functional Requirements**

- FR-01: The system shall allow a new user to register using a unique email address and password. Invalid or incomplete registration data shall be rejected with an appropriate validation message.
- FR-02: The system shall authenticate registered users before allowing access to protected functionality. Invalid credentials shall not grant access and shall result in an appropriate authentication error.

*Each functional requirement is uniquely identified using an FR identifier. Each non-functional requirement uses an NFR identifier and each security requirement uses an SR identifier. Requirements are written as testable statements using measurable or observable expected behaviour wherever applicable.*

- FR-01: The system shall allow a new user to register using a unique email address and password.
  - Acceptance condition: valid registration creates one account; missing/invalid required input is rejected; a duplicate email is not registered twice.
- FR-02: The system shall authenticate registered users before allowing access to protected functionality.
  - Acceptance condition: valid credentials grant authenticated access; invalid credentials are rejected; unauthenticated requests to protected functions are denied.

## **Task Management**

### **5.1.1 Description and Priority**

Provides task creation, viewing, editing, and deletion for authorized users. Priority: High. This feature is the core work-management capability of the application.

### **5.1.2 Stimulus/Response Sequences**

Create: authenticated user submits valid task data → system validates the request and stores the task → system returns the created task or success response. View: user requests authorized tasks → system checks authentication/authorization and returns the permitted task information. Edit: authorized user submits changed task data → system validates and stores the update. Delete: authorized user requests deletion → system verifies permission and removes the task or returns an error.

### **5.1.3 Functional Requirements**

- FR-03: The system shall allow an authenticated user to create a task with a title, description, priority, and deadline where applicable. Invalid required input shall be rejected.
- FR-04: The system shall allow an authenticated user to view tasks that the user is authorized to access.
- FR-05: The system shall allow an authorized user to edit an existing task and shall persist the accepted changes.
- FR-06: The system shall allow an authorized user to delete an existing task and shall prevent unauthorized deletion.

## Task Assignment and Tracking

### 5.1.1 Description and Priority

Provides task assignment, status management, and priority management so that responsibility and work state remain visible. Priority: High.

### 5.1.2 Stimulus/Response Sequences

Assignment: authorized user selects a registered team member for a task → system validates the assignee and permission → assignment is stored and reflected in task details. Status update: authorized user selects a valid status → system validates and stores the new status → task list/dashboard reflects the change. Priority update: authorized user selects a supported priority → system validates and stores the new value.

### 5.1.3 Functional Requirements

- FR-07: The system shall allow an authorized user to assign a task to a registered team member.
  - FR-08: The system shall allow an authorized user to update the status of a task using a valid status value.
  - FR-09: The system shall allow an authorized user to specify and modify task priority using a supported priority value.

## Search, Filtering, and Dashboard

### 5.1.1 Description and Priority

Provides authorized task discovery and progress visibility. Priority: Medium. The feature supports efficient review of work without exposing tasks outside the current user's authorization.

### 5.1.2 Stimulus/Response Sequences

Search/filter: user enters a task attribute or selects a filter → system validates the request and applies it only to authorized task data → matching tasks are displayed. Dashboard: user opens the dashboard → system retrieves authorized task/progress information → system displays task counts and progress summaries such as to-do, in-progress, completed, and overdue tasks where applicable.

### 5.1.3 Functional Requirements

- FR-10: The system shall allow users to search and filter authorized tasks using supported task attributes.
  - FR-11: The system shall display task progress information through a dashboard using authorized task data and consistent progress indicators.

## Other Nonfunctional Requirements

### Performance Requirements

*NFR-01 Performance: Under normal operating conditions, normal task-management requests shall return an application response within 3 seconds for the agreed test environment. The measurement shall be taken from request initiation to receipt of the application response. The requirement applies to representative registration/login and task-management requests and is intended to provide responsive normal operation.*

### Safety Requirements

*The Task Manager Application is not a safety-critical system and does not directly control physical equipment. No physical safety hazard is introduced by normal task-management operation. The software shall nevertheless prevent unsafe software states relevant to data handling by validating input, rejecting malformed requests, and*

*avoiding unintended modification of stored task information. No external safety certification is required for the defined project scope.*

## Security Requirements

### Security Objectives

- Authentication and access control: ensure that only authenticated and appropriately authorized users can access protected task-management functions and task information.
- Protection of sensitive and task data: prevent unauthorized disclosure or modification of user credentials and task information through access checks, secure handling of authentication data, and input validation.

### Security Requirements

- • **SR-01 Authentication:** The system shall require successful authentication before allowing access to protected task-management functions.
- • **SR-02 Authorization:** The system shall enforce authorization checks before allowing a user to view, modify, assign, or delete task information for which the user lacks permission.
- • **SR-03 Password Protection:** The system shall not expose user passwords in application responses or normal application logs and shall use a secure password-handling mechanism selected during implementation.
- • **SR-04 Input Validation:** The system shall validate user-supplied input on the server side before processing registration, authentication, task, search, assignment, and update requests.

## Software Quality Attributes

The following quality and non-functional requirements apply:

- • **NFR-02 Security:** The system shall prevent unauthenticated users from accessing protected task-management functions.
- • **NFR-03 Usability:** The system shall provide a consistent interface for creating, viewing, editing, assigning, and tracking tasks, with understandable labels and validation/error feedback.
- • **NFR-04 Reliability:** The system shall preserve successfully stored user and task information during normal application operation and shall not silently discard a successful update.
- • **NFR-05 Maintainability:** The system shall use modular frontend, backend/service, and data-access components so that an individual component can be modified without requiring unrelated components to be rewritten.
- • **NFR-06 Testability:** The system shall structure major functional components and interfaces so that individual functional behaviour can be tested independently and integration behaviour can be verified through defined APIs.

## Business Rules

### Business Rules

1. A user must be authenticated before accessing protected task-management functions.
2. Only authorized users may modify or delete a task.
3. A task must contain a title before it can be stored.
4. A task shall use a valid status value defined by the application.
5. A task assigned to a user shall reference a registered user.

6. Task priority shall use one of the values supported by the application.
7. Search and filtering shall return only task information the current user is authorized to access.
8. Registration shall not create multiple accounts for the same unique email address.

Domain requirements: the task-management domain shall represent at least the concepts of user, task, priority, status, deadline, assignee, and progress. A task shall contain a title, description where applicable, priority, status, deadline where applicable, and assignee where applicable.

*Domain requirements: the task-management domain shall represent user and task entities and their relevant relationships. Task information shall include a title, description, priority, deadline where applicable, assignee where applicable, status, and ownership/authorization information needed to enforce access rules. User information shall include the identity data necessary for registration and authentication without exposing passwords through normal application interfaces.*

## Other Requirements

*The system shall maintain persistent records for registered users and tasks. Data storage shall support the task CRUD operations and the relationships needed for assignment and authorization. The implementation shall separate presentation, application/service, and data-access responsibilities. The system shall be structured so that future approved enhancements can be added without redesigning unrelated functionality. Internationalization, legal compliance, and external integrations are not separately specified because they are outside the current approved project scope.*

## Appendix A: Glossary

### Glossary terms:

- • **Task:** A unit of work that needs to be completed.
- • **User:** A registered person who interacts with the application.
- • **Team Member:** A registered user who may receive an assigned task.
- • **Priority:** The importance level assigned to a task.
- • **Status:** The current state of a task.
- • **Deadline:** The target date by which a task should be completed.
- • **Assignee:** The registered team member responsible for a task.
- • **Dashboard:** A view that summarizes task and progress information.
- • **Authentication:** Verification of a user's identity.
- • **Authorization:** Determination of the operations and data a user is permitted to access.
- • **API:** Application Programming Interface.
- • **CRUD:** Create, Read, Update, Delete.
- • **FR:** Functional Requirement.
- • **NFR:** Non-Functional Requirement.
- • **SR:** Security Requirement.
- • **SRS:** Software Requirements Specification.
- • **SAD:** Software Architecture and Design Specification.
- • **STP:** Software Test Plan.

- • **RTM:** Requirements Traceability Matrix.

## Appendix B: Field Layouts

Field layouts and properties for the Task Manager Application are provided below. The fields represent the information required to register users and manage tasks. Field lengths are logical validation limits for the application and may be adjusted during implementation if approved through requirements change.

**Task information fields and their properties are listed in the table below. The layout identifies the logical field name, expected data type, purpose, and whether the field is required for the task-management functions.**

| Field       | Length | Data Type                       | Description                                 | Is Mandatory |
|-------------|--------|---------------------------------|---------------------------------------------|--------------|
| Task ID     | 20     | Alphanumeric                    | Unique identifier for a task                | Y            |
| Title       | 100    | Short title describing the task |                                             | Y            |
| Description | 500    | String                          | Detailed description of the task            | N            |
| Priority    | 10     | String                          | Supported priority value for the task       | Y            |
| Deadline    | 10     | Date                            | Target completion date                      | N            |
| Status      | 20     | String                          | Current task state                          | Y            |
| Assignee    | 50     | String                          | Registered team member assigned to the task | N            |
| Created By  | 50     | String                          | Registered user who created/owns the task   | Y            |

**Task and progress report requirements are listed below. A task report shall provide task-level information needed for review, while a progress report shall summarize task counts and completion information for authorized data.**

| Task Report         | Progress Report     |
|---------------------|---------------------|
| Task ID             | Total Tasks         |
| Title               | To Do Tasks         |
| Priority            | In Progress Tasks   |
| Deadline            | Completed Tasks     |
| Status              | Overdue Tasks       |
| Assignee            | Progress Percentage |
| Created By          | Tasks by Priority   |
| Authorization Scope | Tasks by Assignee   |

## Appendix C: Requirement Traceability Matrix

| Sl. No | Requirement ID | Brief Description of Requirement | Architecture Reference                                   | Design Reference   | Code File Reference   | Test Case ID | System Test Case ID |
|--------|----------------|----------------------------------|----------------------------------------------------------|--------------------|-----------------------|--------------|---------------------|
| 1      | FR-01          | User registration                | SAD 3.3 Authentication Module; 3.4 Security Architecture | SAD 4.2 API Design | Part 2 implementation | TC-01        | Part 2              |

|    |        |                     |                                                          |                                              |                       |       |        |
|----|--------|---------------------|----------------------------------------------------------|----------------------------------------------|-----------------------|-------|--------|
| 2  | FR-02  | User authentication | SAD 3.3 Authentication Module; 3.4 Security Architecture | SAD 4.1 Login Sequence; 4.2 API Design       | Part 2 implementation | TC-02 | Part 2 |
| 3  | FR-03  | Create task         | SAD 3.3 Task Management Module                           | SAD 4.2 API Design; 4.1 Create Task Sequence | Part 2 implementation | TC-03 | Part 2 |
| 4  | FR-04  | View tasks          | SAD 3.3 Task Management Module                           | SAD 4.2 API Design                           | Part 2 implementation | TC-04 | Part 2 |
| 5  | FR-05  | Edit task           | SAD 3.3 Task Management Module                           | SAD 4.2 API Design                           | Part 2 implementation | TC-05 | Part 2 |
| 6  | FR-06  | Delete task         | SAD 3.3 Task Management Module                           | SAD 4.2 API Design                           | Part 2 implementation | TC-06 | Part 2 |
| 7  | FR-07  | Assign task         | SAD 3.3 Task Management Module                           | SAD 4.2 API Design                           | Part 2 implementation | TC-07 | Part 2 |
| 8  | FR-08  | Update task status  | SAD 3.3 Task Management Module                           | SAD 4.2 API Design                           | Part 2 implementation | TC-07 | Part 2 |
| 9  | FR-09  | Task priority       | SAD 3.3 Task Management Module                           | SAD 4.2 API Design                           | Part 2 implementation | TC-07 | Part 2 |
| 10 | FR-10  | Search and filter   | SAD 3.3 Task Management Module; 3.3 Dashboard Module     | SAD 4.2 API Design                           | Part 2 implementation | TC-08 | Part 2 |
| 11 | FR-11  | Dashboard           | SAD 3.3 Dashboard Module                                 | SAD 4.5 UX Design                            | Part 2 implementation | TC-09 | Part 2 |
| 12 | NFR-01 | Performance         | SAD 3.1 Architecture Goals & Constraints                 | SAD 4.4 Error Handling/Monitoring            | Part 2 implementation | TC-10 | Part 2 |
| 13 | NFR-02 | Security            | SAD 3.4 Security Architecture                            | SAD 4.4 Error Handling                       | Part 2 implementation | TC-10 | Part 2 |
| 14 | NFR-03 | Usability           | SAD 3.2 Stakeholders & Concerns                          | SAD 4.5 UX Design                            | Part 2 implementation | TC-09 | Part 2 |
| 15 | NFR-04 | Reliability         | SAD 3.1 Architecture Goals & Constraints                 | SAD 4.4 Error Handling/Monitoring            | Part 2 implementation | TC-10 | Part 2 |

|    |        |                                     |                                                                      |                                        |                       |              |        |
|----|--------|-------------------------------------|----------------------------------------------------------------------|----------------------------------------|-----------------------|--------------|--------|
| 16 | NFR-05 | Maintainability                     | SAD 3.1 Architecture Goals & Constraints; 3.3 Component Architecture | SAD 4.3 Design Decisions               | Part 2 implementation | TC-10        | Part 2 |
| 17 | NFR-06 | Testability                         | SAD 3.1 Architecture Goals & Constraints                             | SAD 4.2 API Design; 4.4 Error Handling | Part 2 implementation | TC-10        | Part 2 |
| 18 | SR-01  | Authentication security requirement | SAD 3.4 Security Architecture                                        | SAD 4.1 Login Sequence; 4.2 API Design | Part 2 implementation | TC-02, TC-10 | Part 2 |
| 19 | SR-02  | Authorization security requirement  | SAD 3.4 Security Architecture                                        | SAD 4.2 API Design                     | Part 2 implementation | TC-07, TC-10 | Part 2 |
| 20 | SR-03  | Password protection                 | SAD 3.4 Security Architecture                                        | SAD 4.4 Error Handling/Monitoring      | Part 2 implementation | TC-10        | Part 2 |
| 21 | SR-04  | Input validation                    | SAD 3.4 Security Architecture                                        | SAD 4.2 API Design; 4.4 Error Handling | Part 2 implementation | TC-03, TC-10 | Part 2 |